

TEST YOURSELF: KNOWLEDGE VERIFICATION



UNIVERSIDAD

Escuela Colombiana de Ingeniería Julio Garavito

Project: Lab 05 - Reverse Engineering (Questionnaire)

Subject: IT Security and Privacy

Professor: Daniel Esteban Vela López

Authors: Andersson David Sánchez Méndez & Cristian Santiago Pedraza Rodríguez

Date: 2026-02-24

Location: Bogotá, Colombia

Confidentiality: Internal / Academic Use Only



1. Conceptual: Disassembler vs. Debugger

1.1 Main Differences

The fundamental difference lies in the **execution state** of the target program:

- ♦ **Disassembler** (e.g., `objdump`, `Ghidra`, `Cutter` in static mode): It performs **Static Analysis**. It translates machine code (bytes) back into assembly language without executing the file. It provides a "dead" map of the code's structure.
- ♦ **Debugger** (e.g., `GDB`): It performs **Dynamic Analysis**. It runs the program in a controlled environment. It allows the analyst to pause execution (breakpoints), step through instructions one by one, and inspect the "live" state of memory and CPU registers.

1.2 Use Cases

- ♦ **Use a Disassembler when:** You need to understand the global logic of the program, find hardcoded strings (like the `"7Wtyr"` password), or identify vulnerabilities without the risk of executing potentially malicious code.
- ♦ **Use a Debugger when:** You need to see the exact values being compared in registers (like `$rdi` and `$rsi` in this lab), trace a specific execution path that depends on complex calculations, or bypass checks in real-time.



2. Practice: Analysis Methodology for Unknown Binaries

Based on our experience with the `passguess` binary, the step-by-step methodology is:

1. **Initial Triage (`file`):** Identify the binary's DNA. Is it 64-bit? Is it stripped? (e.g., we found `passguess` was a non-stripped ELF x86-64).
2. **Surface Reconnaissance (`strings`):** Look for "low-hanging fruit." UI messages like `"[OK] Password Found"` or suspicious literals like `"7Wtyr"` provide immediate clues about the program's intent.
3. **Static Logic Mapping (`Ghidra` / `objdump`):** Decompile the binary to understand the high-level flow. Identify the authentication function and the specific comparison call (e.g., `strcmp`).
4. **Visual Flow Analysis (`Cutter`):** Use graph views to identify the "Decision Points"—the conditional jumps (`je` / `jne`) that act as gatekeepers for success or failure paths.
5. **Dynamic Verification (`GDB`):** Verify the theory by setting breakpoints at the comparison address. Inspecting `$rsi` during the `strcmp` call confirms the live secret.
6. **Manipulation (`radare2`):** Once the gatekeeper is identified, apply a patch (like inverting a `jne` to `je`) to verify the exploit.



3. Ethics: Legal and Moral Boundaries

3.1 Legal Circumstances

Reverse engineering commercial software is not a "legal binary"; it depends heavily on jurisdiction (e.g., DMCA in the US, EU Directives), but is generally protected for:

- ◆ **Interoperability:** Discovering how to make new software work with an existing proprietary system.
- ◆ **Security Research:** Finding vulnerabilities to improve security (often under "Fair Use" or Bug Bounty programs).
- ◆ **Archival/Fixes:** Correcting a critical bug in software that is no longer supported by the vendor.

3.2 Required Precautions

An ethical analyst must always maintain a "Clean Room" approach:

- ◆ **Isolation:** Use Virtual Machines (VMs) to ensure no interference with the host system.
- ◆ **Non-Distribution:** extracted keys, "cracks," or proprietary algorithms must not be shared or sold.
- ◆ **Responsible Disclosure:** If a vulnerability is found in a commercial product, the vendor should be notified privately before any public disclosure.



4. Final Reflection

Why is hardcoding passwords insecure?

As demonstrated in this laboratory, a binary is not a "black box." Hardcoding secrets like `"7Wtyr"` is equivalent to leaving a key under a transparent doormat. Any analyst with basic tools can extract the string in seconds.

Proposed Alternative:

A secure system should never store the password itself. Instead, it should store a **salted cryptographic hash** (e.g., using Argon2 or bcrypt). The program would then hash the user's input and compare the two hashes. This way, even if the binary is reversed, the analyst only finds a one-way hash, not the plaintext password.

